PHP

Profª. Dra. Andréia Rodrigues Casare
E-mail: casareandreia@gmail.com
andreia.casare@cps.sp.gov.br

PHP

- A linguagem PHP pode ser usada de duas formas: incorporado ao HTML ou de forma “pura”, em scripts separados das páginas HTML.
- Tudo que estiver dentro dessas tag's PHP será considerado script PHP, e não HTML.
- Código PHP digitado fora das tag's será interpretado como erro.
- ***Obs:*** Cada linha de código PHP deve terminar com “;” para que a linguagem reconheça o fim de uma linha de comando e o começo de uma nova.

COMENTÁRIOS

- **Comentários de uma linha**
- <?php

  //este é um comentário de uma linha

  #este também é um comentário de uma linha

  ?>

- **Comentários de várias linhas**

  <?php

  /*Este é um exemplo de comentário que ocupa mais de uma linha. Aqui eu não preciso colocar o delimitador em todas as linhas. */

  ?>

VARIÁVEIS EM PHP

- Sempre começam com $ (cifrão)
- Não precisam declarar tipo (PHP é dinamicamente tipado)
- O nome pode conter letras, números e _

VARIÁVEIS EM PHP

- **Exemplo de nomes válidos:**
  - $nome, $endereco, $idade, $data_inscricao
  - PHP é Case Sensitive, ou seja, diferencia maiúsculas e minúsculas. A variável $a é diferente da variável $A.

TIPOS DE VARIÁVEIS

- String → "texto"
- Integer → 123
- Float (double) → 3.14
- Boolean → true / false
- Array → lista de valores
- Object → usado com classes

Exemplo do uso de variáveis

```
<?php
  $nome = "Andréia";   // String
  $idade = 25;         // Inteiro
  $altura = 1.68;      // Float (número decimal)
  $estudando = true;   // Booleano (verdadeiro/falso)

  echo $nome;          // Mostra na tela: Andréia
?>
```

VARIÁVEIS SUPERGLOBAIS

- PHP possui várias variáveis que podem ser acessadas de qualquer lugar do script. Exemplos:

- $_GET, $_POST, $_SESSION, $_COOKIE, e $_SERVER.

EXIBINDO O CONTEÚDO NA TELA (echo)

- Podemos exibir o conteúdo de uma variável na tela com o uso do comando echo.
  - $nome=”Andréia”;
  - echo “Professora $nome <tag>”;
- Também podemos utilizar apóstrofos no lugar de aspas, porém, apóstrofos mostram o nome de uma variável em vez de seu conteúdo.

EXIBINDO O CONTEÚDO NA TELA (echo)

- Aspas duplas (") - Interpretam variáveis e caracteres especiais (como \n, \t etc).
- Aspas simples (') - Mostram o conteúdo exatamente como está escrito (texto literal), sem interpretar variáveis.

OPERAÇÕES ARITMÉTICAS

Operação	Operador	Exemplo	Resultado
adição	+	$a + $b	6
subtração	-	$a - $b	2
multiplicação	*	$a * $b	8
divisão	/	$a / $b	2
módulo (resto da divisão)	%	$a % $b	0

OPERAÇÕES COM STRINGS E ATRIBUIÇÃO

- **Operador “.”**
- Utilizamos o operador “.” para concatenar (unir) dois valores (geralmente Strings).
- Ex: quando queremos unir o valor de uma variável a uma string qualquer.
  - echo “Meu nome é “.$nome.” e moro em “.$cidade.” desde ”.$data.”.”;

OPERAÇÕES COM STRINGS E ATRIBUIÇÃO

- **Operador ".="**
- Utilizamos operador de atribuição para adicionar a uma variável um novo conteúdo, sem que o conteúdo antigo seja perdido. Ex: Atribuímos à variável $nome o nome do usuário.
- $nome = "Leopoldo";
- O conteúdo da variável é "Leopoldo". Agora, queremos adicionar o sobrenome à variável, mas devemos fazer isso sem que o nome seja perdido. Para isso, utilizamos o operador ".=", que tem a função de atribuir o valor à variável, sem que seu conteúdo seja perdido.
- $nome .= " da Silva";

OPERADORES LÓGICOS

Operador	Operação
&&	E lógico
||	Ou lógico
!	negação

OPERADORES DE COMPARAÇÃO

Operação	operador	Sintaxe
igual	==	$a == $b //retorna verdadeiro se forem iguais
diferente	!=	$a != $b //retorna verdadeiro se forem diferentes
maior/menor	< ou >	$a > $b //verdadeiro se $a maior que $b
maior ou igual/menor ou igual	>= ou <=	$a >= $b //verdadeiro se $a for maior ou igual a $b

ESTRUTURA IF

- A estrutura if (“se”) executa um determinado bloco, caso uma condição seja verdadeira. Sua sintaxe é:

```
<?php
if(condição){
    comandos;
}
?>
```

- A utilização dos delimitadores “{“ e “}” é obrigatória, exceto se o bloco contiver apenas uma linha de código. Ex:

```
<?php
if($valor1 == 2){
     echo “valor da variável é 2”;
}
?>
```

ESTRUTURA IF

- Essa estrutura executa um determinado bloco caso a condição seja verdadeira e outro caso seja falsa.

```
<?php
if($a % 2 == 0){
    echo “Valor da variável: “.$a;
    echo “Variável é um número par!”;
}else{
    echo “Valor da variável: “.$a;
    echo “Variável é ímpar!”;
}
?>
```

ESTRUTURA WHILE

- A estrutura while é uma estrutura de repetição. Ela avalia a expressão e executa o bloco de comandos enquanto a condição for verdadeira. caso a condição retorne falsa, o while deixa de executá-la. Sua sintaxe é:

```
<?php
while(expressão){
   comandos;
}
?>
```

ESTRUTURA WHILE

- Devemos ter um cuidado especial a utilizar estruturas de repetição, como while.
- Temos que ter certeza de que há uma chance da expressão avaliada retornar o valor Falso, do contrário, irá se tornar um laço infinito, ou seja, não vai parar de executar. Ex:

```
<?php
$cont = 1;
while($cont < 10){
    echo “Valor de \$cont: $cont<br>”;
    $cont++;
}
?>
```

ESTRUTURA DO...WHILE

- A estrutura do...while é semelhante a while, porém, enquanto a estrutura while avalia a condição antes de executar, a estrutura do...while executa o bloco antes de fazer o teste. Ex:

```
<?php
$cont = 1;
do{
   echo “Valor de \$cont: $cont<br>”;
   $cont++;
}while($cont < 10);
?>
```

ESTRUTURA FOR

- A utilização do for é idêntica a do while, entretanto, sua sintaxe é diferenciada. Nele, definimos uma variável contadora, uma condição para o final da repetição e uma regra de modificação do contador.

```
<?php
for(inicialização;condição;operador){
   comandos
}
?>
```

ESTRUTURA FOR

- Na inicialização, definimos e inicializamos o contador. Depois, criamos uma condição para o término da execução do for. Em seguida definimos o parâmetro de atualização da variável de controle do for. Ex:

```
<?php
for($cont=0;$cont<10;$cont++){
    echo “Valor da variável \$cont é $cont<br>”;
}
?>
```

EXERCÍCIOS

- Faça um programa em PHP que receba 3 notas de um aluno e calcule a média. Mostre a tela a mensagem:
- Média abaixo de 4 → "Reprovado"
- Média entre 4 e 6.9 → "Recuperação"
- Média 7 ou mais → "Aprovado"

ARRAYS

- Array (ou Vetor) é um conjunto ordenado de variáveis.
- Os arrays possuem um índice, que pode ser um número ou texto.
- Ex: $lista[] ou $lista[0].
- No PHP, os arrays iniciam-se em zero. Diferente de outras linguagens, você não precisa definir um número limite de posições dentro de um array, pois ele se expande dinamicamente.
- Caso o índice não seja indicado entre os colchetes, o PHP irá procurar pelo último elemento do array e atribuir o novo valor na posição seguinte.

ARRAYS

- $lista[]; //isso indica que a variável lista é um array
- $lista[0] = 10; //valor 10 na posição zero; a primeira posição do array
- $lista[1] = 11; //valor 11 na posição 1
- $lista[] = 12; //valor 12 é colocado logo em seguida, na posição 2
- $lista[] = 13; //valor 13 colocado na posição 3
- $lista[7] = 14; //valor 14 colocado na posição 7, deixando as anteriores vazias
- $lista[] = 15; //valor 15 colocado na posição 8 do array

ARRAYS

- Podemos utilizar a função array para atribuir valores a um vetor. Os valores devem estar entre parênteses e separados por vírgula. Ex:
- $lista = array(10, 11, 12, 13, 14, 15);
- Também é possível utilizar nomes em vez de números para referenciar um array, criando assim um array associativo. Ex:
- $usuario['nome'] = “João”;
- $usuario['idade'] = 25;
- $usuario['cidade'] = “Guaira”;

ESTRUTURA FOREACH

- **Oferece uma maneira mais simples de percorrer os elementos de um array. Possui duas sintaxes:**

```
<?php
foreach($array as $elemento){
    comandos;
}
?>
```

ou

```
<?php
foreach($array as $chave => $valor){
    comandos;
}
?>
```

ESTRUTURA FOREACH - EXEMPLOS

```
<?php
$valor = array(5, 2, 7, 4);
foreach($valor as $v){
    echo “Valor é: $v <br>”;
}
?>
```

```
<?php
$pessoa = ["nome" => "André", "idade" =>30, "cidade" => "São Paulo"];

foreach ($pessoa as $chave => $valor) {
    echo "$chave: $valor<br>";
}
?>
```

EXEMPLO

```
<?php
  $nomes = array('Edgar', 'Maria', 'Carlos');
  for ($i = 0; $i < 3; $i++) {
     echo 'Posição: ' . $i . ' Nome: ' . $nomes[$i] . '<br />';
}
```

O mesmo código usando o foreach:

```
<?php
   $nomes = array('Edgar', 'Maria', 'Carlos');
   foreach ($nomes as $posicao => $nome) {
   echo 'Posição: ' . $posicao . ' Nome: ' . $nome . '<br >';
 }
```

ESTRUTURA SWITCH

- O comando switch testa linha a linha os cases encontrados, e a partir do momento que encontra um valor igual ao da variável testada, passa a executar todos os comandos seguintes, mesmo os que fazem parte de outro teste, até o fim do bloco.
- Por isso usa-se o comando break, quebrando o fluxo e fazendo com que o código seja executado da maneira desejada.

ESTRUTURA SWITCH - SINTAXE

```
<?php
switch(condicao){
   case valor1 : comando1
            break;
   case valor2 : comando2
            break;
   case valorn : comandon
            break;
   default: comando_default
            break;
  }
?>
```

EXEMPLO

```
<?
switch($cor){
   case 'vermelho' : echo “cor vermelha<br>”;
            break;
   case 'verde' : echo “cor verde<br>”;
            break;
   case 'azul' : echo “cor azul<br>”;
            break;
   default : echo “escolha uma cor<br>”;
            break;
}
?>
```

COMANDO CONTINUE

- O comando continue também deve ser utilizado no interior de laços, e funciona de maneira semelhante ao break, com a diferença que o fluxo ao invés de sair do laço volta para o início dele.
- **Exemplo:**

```
<?
for ($i = 0; $i < 100; $i++) {
    if ($i % 2)
        continue;
echo " $i ";
}
?>
```

EXERCÍCIOS

- Crie um script PHP usando FOR e FOREACH de 1 a 12 que exiba na tela os meses de Janeiro a Dezembro. (pode ser usado array)

FUNÇÕES

- Funções são blocos de códigos destinados a executar uma determinada tarefa.
- Sintaxe:

```
<?php
function nome_funcao(arg_1, arg_2,arg_n){
    comandos;
    return  valor;
}
?>
```

EXEMPLO 1

```
<?php
function soma(){
    $num1=10;
    $num2=7;
    $resu= $num1 + $num2;
    echo “A soma é: $resu”;
 }
?>
```

CHAMANDO A FUNÇÃO

```
<?php
function soma(){
    $num1=10;
    $num2=7;
    $total= $num1 + $num2;
    echo "<br>A soma é: $total";
 }

echo "<br>Chamando a função";
soma();
?>
```

EXEMPLO 2

```
<?php
function soma_pares(){
        $pares=0;
        for($i=2;$i<=100;$i+=2){
                $pares+=$i;
        }
   echo "<br> Soma dos pares entre 2 e 100: $pares";
 }
echo "<br>Chamando a função";
soma_pares();
?>
```

VALOR DE RETORNO

- Uma função pode ou não retornar um valor. Para isso é necessário usar o comando return no final da função, indicando o valor que será retornado.

```
<?php
function soma_pares(){
    $pares=0;
    for($i=2;$i<=100;$i+=2){
            $pares+=$i;
    }
    return $pares;
}
$resu=soma_pares();
echo "<br> Soma dos pares entre 2 e 100: $resu";
?>
```

OUTRO EXEMPLO

```
<?php
function soma(){
    $num1=10;
    $num2=7;
    $total= $num1 + $num2;
    return $total;
}

echo "<br>Chamando a função";
echo "<br>A soma é: ".soma();
```

ARGUMENTOS OU PARÂMETROS

- São valores passados à função para serem manipulados pela mesma. A passagem de parâmetros não é obrigatória. Os parâmetros devem ser declarados entre parênteses, após o nome da função. Os parâmetros serão recebidos de acordo com a ordem de passagem e transformados em variáveis locais para que a função possa utilizá-los.

EXEMPLO DE FUNÇÕES PASSANDO PARÂMETROS

```
<?php
function soma($num1,$num2){
    $total= $num1 + $num2;
    echo "<br>A soma é: ".$total;
}

echo "<br>Chamando a função e passando parâmetros";
soma(10,8);
?>
```

EXEMPLO DE FUNÇÕES PASSANDO PARÂMETROS

```
function perfeito($num){
        $soma=0;
        for($i=1;$i<$num;$i++){
                if ($num % $i == 0){
                        $soma+=$i;  }
        }
        if ($soma == $num){
                $msg=" Perfeito ";
        }else{
                $msg=" Não Perfeito ";  }
        return $msg;
}
echo "<br>Chamando a função e passando parâmetros";
$n=6;
$resu=perfeito($n);
echo "<br> O número $n é: $resu";
```

PASSAGEM DE PARÂMETROS POR REFERÊNCIA

- Os exemplos anteriores mostram a passagem de parâmetros por valor, ou seja, o valor da variável é passado à função, porém, quando a função termina de executar, o valor da variável passada à função como parâmetro permanece inalterado. Para que uma função possa alterar o valor de uma variável (ou mais de uma), é necessário fazer a passagem de parâmetros por referência, ou seja, a função irá receber a própria variável como parâmetro e poderá assim alterar seu valor.

EXEMPLO PASSAGEM DE PARÂMETROS POR VALOR

```
<?php
function soma10($num){
    $num += 10;
}
$a = 5;
soma10($a);
echo $a;
?>
```

- O resultado do script acima será 5, pois houve uma passagem de parâmetro por valor para a função soma10(). Neste caso, o valor de $a foi "copiado" e enviado à função, porém, o conteúdo de $a foi mantido pelo PHP.

PASSAGEM DE PARÂMETROS POR REFERÊNCIA

- Para que a função possa alterar o valor de $a, a variável deve ser passada por referência, ou seja, o endereço de memória da variável deve ser passado para a função (afinal, variáveis são endereços de memória que armazenam determinado valor). A passagem por referência é feita utilizando o sinal “&”.

EXEMPLO PASSAGEM DE PARÂMETROS POR REFERÊNCIA

```
<?php
function soma10(&$num){ //O & antes de $num
   indica passagem por referência
$num += 10;
}
$a = 5;
soma10($a);
echo $a;
?>
```

- **O resultado agora será 15.**

EXERCICIOS

- Faça um script em php que simule uma calculadora usando função. Receba 2 números por parâmetro e opção (1-soma, 2-subtração,3-multiplicação, 4-divisão).

- Faça um script em php que, receba um número, calcule o fatorial usando função e exiba o resultado dentro da função.

- Crie uma função que receba dois números inteiros por parâmetro e retorne a soma dos N números inteiros existentes entre esses dois números (inclusive). Imprima o resultado no local que chamou a função.

FATORIAL FUNÇÃO/RECURSIVIDADE

```
function fatorial($numero){
	if($numero <= 1){
		return $numero;
	}else{
		return $numero * fatorial($numero - 1);
	}
}
```

Conectando HTML COM PHP

```html
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cadastro de Clientes</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <h1>Cadastro de Clientes</h1>
    </header>
    <main>
        <section class="formulario">
            <h2>Adicionar Novo Cliente</h2>
            <form action="cadastrar.php" method="POST">
                <label for="nome">Nome:</label>
                <input type="text" id="nome" name="nome" required>
                <label for="email">E-mail:</label>
                <input type="email" id="email" name="email" required>
                <label for="telefone">Telefone:</label>
                <input type="text" id="telefone" name="telefone" placeholder="(99) 99999-9999">
                <label for="endereco">Endereço:</label>
                <input type="text" id="endereco" name="endereco">

                <label for="tipo_cliente">Tipo de Cliente:</label>
                <select id="tipo_cliente" name="tipo_cliente" required>
                    <option value="">Selecione</option>
                    <option value="pf">Pessoa Física</option>
                    <option value="pj">Pessoa Jurídica</option>
                </select>
```

Cadastrar.php

```
<?php
  $nome       = $_POST['nome'] ?? null;
  $email      = $_POST['email'] ?? null;
  $telefone   = $_POST['telefone'] ?? null;
  $endereco   = $_POST['endereco'] ?? null;
  $tipo_cliente = $_POST['tipo_cliente'] ?? null;
  $sexo       = $_POST['sexo'] ?? null;

 Echo ‘Cliente: ‘.$nome.’ mora em ‘.$endereco.’<br>’;
 Echo “<br>”;
 If ($tipo_cliente == ‘pf’){
     Echo “Pessoa Física”;
 }else{
    Echo “Pessoa Jurídica”;
 }
?>
```